BSCS23161

DSA - Assignment #03

1)

i) Multiplication

```
int RecMultip(int a, int b)
{  if(b==0) return 0;
   return a + RecMultip(a,b-1);
}
```

Dry Run:-

e.g RecMultip(2,4)

- 4!=0 so return 2+ RecMultip(2,4-1=3)
- 3!=0 so // // // // (2,3-1=2)
- 2!=0 // // // // // (2,2-1=1)
- 1!=0 // // // // // (2,1-1=0)
- 0==0 so return 0 and backtrack
- 2+RecMultip(2,0) = 2+0=2
- // // // (2,1) = 2+2=4
- (2,2) = 2+4=6
- (2,3) = 2+6=8
- return 8

Call Stack

0 ←	RecM(2,0)
2 ←	RecM(2,1)
4 ←	RecM(2,2)
6 ←	RecM(2,3)
8 ←	RecM(2,4)

Recursive Tree

RecM(2,4) 8 ↩
RecM(2,3) 6 ↩
RecM(2,2) 4 ↩
RecM(2,1) 2 ↩
RecM(2,0) → 0

Time Complexity:-
Fn calls itself recursively b times because in each call, b is decrement by till b==0. So it is O(b)

ii) Fast Multiplication

```
int FastRecMultip(int a, int b)
{  if(b==0) return 0;
   if(b%2==0)
       return FastRecMultip(a+a, b/2);
   else
       return a+FastRecMultip(a+a,(b-1)/2);
}
```

Dry Run :-

e.g FastRecMultip(2, 4)

- 4!=0 but 4%2=0 so return FRM(2+2, 4/2)
- 2!=0 but 2%2=0 so // // (4+4, 2/2)
- 1!=0 but 1%2!=0 so // 8+FRM(8+8, (1-1)/2)
- 0==0 so return 0. Backtrack
- 8+0=8 → return 8 -> return 8

* Hence FRM(2,4) = 8

Call stack

FRM(16,0)	-> 0
FRM(8,1)	-> 8
FRM(4,2)	-> 8
FRM(2,4)	-> 8

Recursive Tree

FRM(2,4) 8
- FRM(4,2) 8
  - FRM(8,1) 8
    - FRM(16,0) -> 0

Time complexity :-

In each recursive call, b is divided by 2. So time complexity is O(log b).

2)

i) Power

```
int RecPow(int b, int p)
{ if (p==0) return 1;
  return b * RecPow(b, p-1);
}
```

Dry run :-

e.g RecPow(2, 4)

- 4!=0 so return 2 * RecPow(2,3)
- 3!=0 " " " " " " (2,2)
- 2!=0 (2,1)
- 1!=0 (2,0)
- 0==0 so return 1 and backtrack
- RecPow(2,1) = 2x1 = 2
  (2,2) = 2x2 = 4
  (2,3) = 2x4 = 8
  (2,4) = 2x8 = 16

Call Stack

RecPow(2,4)
RecPow(2,3)
RecPow

RecPow(2,0)	->1
RecPow(2,1)	->2
RecPow(2,2)	->4
RecPow(2,3)	->8
RecPow(2,4)	->16

Recursive Tree

RecPow(2,4) 16
`RecPow(2,3) 8
`RecPow(2,2) 4
`RecPow(2,1) 2
`RecPow(2,0) -> 1

Time complexity:-
:: Fn recursively calls itself p times until it reaches base case p=0, it is O(p)

ii) FastPower

```
int FRP(int a, int b)
{ if(b==0) return 1;
  if (b%2==0)
    return FRP(a*a, b/2);
  else
    return a * FRP(a*a, (b-1)/2);
}
```

Dry run:
e.g FRPC 2,4)
. 4!=0 but 4%2==0 so return FRP(2*2, 4/2)
. 2!=0 but 2 is even so return FRP(4*4, 1)

256 0
• 1!=0 but 1 is odd so return 16*FRP(16*16, 1-1/2
1 0≡0 return 1 and backtrack
• FRP(16,1) = 16
FRP(4,2) returns 16
FRP(2,4) returns 16.

Recursive Tree

FRP(2,4) 16
↳ FRP(4, 2) 16
↳ FRP(16,1) 16
↳ FRP(256,0) → 1

Call stack

FRP(256,0)	→ 1
FRP(16,1)	→ 16
FRP(4,2)	→ 16
FRP(2,4)	→ 16

Time complexity:
• In each recursive call, b is divided by 2. Thus it has O(log b)

5) Bubble Sort without loops

```
bool BSscan(vector<int>&myVec, int ind)
{ if(ind+1==myvec.size()) return false;
  bool ChangHap=false;
  if(myVec[ind]>myVec[ind+1])
    { swap(myvec[ind],myVec[ind+1]);
      ChangeHap=true;
    }
  return BSscan(myVec,ind+1)||ChangHap;
}
void BSwOLoops(vector<int>&myvec)
{ bool ChangHap=BSscan(myVec,0);
  if(ChangHap)
     BSWOLoops(myvec);
  else
     return;
}
int main()
{ vector<int> myVec{4,2,1,9};
  BSWOLoops(myVec);
  print(myVec);
  return 0;
}
```

Dry Run:

e.g myVec = {4, 2, 1, 9}

∴ Call to BSWOLoops(myVec)

- Call to BSScan(myVec, 0)
  - 4 > 2 = True. Swap. ChaHap = T. {2, 4, 1, 9}
  - call to BSScan(myVec, 1)
  - 4 > 1 = T. Swap. ChaHap = T. {2, 1, 4, 9}
- Call to BSScan(myVec, 2)
  - 4 > 9 = F. No swap. ChanHap = F
- Call to BSScan(myVec, 3)
  - Base case so return F. Backtrack
- Return T bec Swap occurred
- return T " " " .
  " " T " " " .

∴ Since overall, ChaHap = T, again call to BSWOLoops(myVec)

- Call to BSScan(myVec, 0):
  - 2 > 1 = T. Swap. ChaHap = T. {1, 2, 4, 9}
- Call to BSScan(myVec, 1):
  - 2 > 4 = F. No swap. ChaHap = F
- Call to BSScan(myVec, 2)
  - 4 > 9 = F. No swap
- Call to BSScan(myVec, 3)
  - base case → return false. Back track

- return false
- return T because swap occured
- " " " " "
- Because ChaHap = T, call to BSWO loops (myVec) again.
- Call to BSScan (myVec, 0)
  - 1 > 2 = F NO swap
- Call to BSScan (myVec, 1)
  - 2 > 4 = F. No swap
- Call (myVec, 2)
  - 4 > 9 = F. NO swap
- Call (myVec, 3)
  - base case → return false. Backtrack
- return F
- return F
- ChanHap = False so BSWO (myVec) not called again.

BSCS23161

DSA - Assignment #03

1)

i) Multiplication

```
int RecMultip(int a, int b)
{  if(b==0) return 0;
   return a + RecMultip(a,b-1);
}
```

Dry Run:-

e.g RecMultip(2,4)

- 4!=0 so return 2 + RecMultip(2,4-1=3)
- 3!=0 so " " " (2,3-1=2)
- 2!=0 " " " " " (2,2-1=1)
- 1!=0 " " " " " (2,1-1=0)
- 0==0 so return 0 and backtrack
- 2 + RecMultip(2,0) = 2+0=2
- " " " (2,1) = 2+2=4
- (2,2) = 2+4=6
- (2,3) = 2+6=8
- return 8

Call Stack

0 ←	RecM(2,0)
2 ←	RecM(2,1)
4 ←	RecM(2,2)
6 ←	RecM(2,3)
8 ←	RecM(2,4)

Recursive Tree

RecM(2,4) 8 ↩
- RecM(2,3) 6 ↩
  - RecM(2,2) 4 ↩
    - RecM(2,1) 2 ↩
      - RecM(2,0) → 0

Time Complexity:-

Fn calls itself recursively b times because in each call, b is decrement by till b==0. So it is O(b)

ii) Fast Multiplication

```
int FastRecMultip(int a, int b)
{   if(b==0) return 0;
    if(b%2==0)
        return FastRecMultip(a+a, b/2);
    else
        return a+FastRecMultip(a+a,(b-1)/2);
}
```

Dry Run :-

e.g FastRecMultip(2,4)

- 4!=0 but 4%2=0 so return FRM(2+2, 4/2)
- 2!=0 but 2%2=0 so // // (4+4, 2/2)
- 1!=0 but 1%2!=0 so // 8+FRM(8+8, (1-1)/2)
- 0==0 so return 0. Backtrack
- 8+0=8 ~~8+8~~ return 8 -> return 8

* Hence FRM(2,4) = 8

Call stack

FRM(16,0)	-> 0
FRM(8,1)	-> 8
FRM(4,2)	-> 8
FRM(2,4)	-> 8

Recursive Tree

FRM(2,4) 8
- FRM(4,2) 8
  - FRM(8,1) 8
    - FRM(16,0) -> 0

Time complexity :-

In each recursive call, b is divided by 2. So time complexity is O(log b).

2)

i) Power

```
int RecPow(int b, int p)
{ if (p==0) return 1;
  return b * RecPow(b, p-1);
}
```

Dry run :-

e.g RecPow(2, 4)

- 4!=0 so return 2 * RecPow(2, 3)
- 3!=0 " " " " " (2, 2)
- 2!=0 (2, 1)
- 1!=0 (2, 0)
- 0==0 so return 1 and backtrack
- RecPow(2,1) = 2x1 = 2
  (2,2) = 2x2 = 4
  (2,3) = 2x4 = 8
  (2,4) = 2x8 = 16

Call Stack

~~RecPow(2,4)~~
~~RecPow(2,3)~~
~~RecPow~~

RecPow(2,0)	→1
RecPow(2,1)	→2
RecPow(2,2)	→4
RecPow(2,3)	→8
RecPow(2,4)	→16

Recursive Tree

RecPow(2,4) 16
`RecPow(2,3) 8
`RecPow(2,2) 4
`RecPow(2,1) 2
`RecPow(2,0) -> 1

Time complexity:-
:: Fn recursively calls itself p times until it reaches base case p=0, it is O(p)

ii) FastPower

```
int FRP(int a, int b)
{ if(b==0) return 1;
  if (b%2==0)
     return FRP(a*a, b/2);
  else
     return a * FRP(a*a, (b-1)/2);
}
```

Dry run:
e.g FRPC 2,4)
. 4!=0 but 4%2==0 so return FRP(2*2, 4/2)
. 2!=0 but 2 is even so return ERP(4*4, 1)

• 1!=0 but 1 is odd so return 16*FRP(16*16, 1-1)
256 0
• 0≡0 return 1 and backtrack
• FRP(16,1) = 16
FRP(4,2) returns 16
FRP(2,4) returns 16.

Recursive Tree

FRP(2,4) 16
↳ FRP(4,2) 16
↳ FRP(16,1) 16
↳ FRP(256,0) → 1

Call stack

FRP(256,0)	→ 1
FRP(16,1)	→ 16
FRP(4,2)	→ 16
FRP(2,4)	→ 16

Time complexity:
• In each recursive call, b is divided by 2. Thus it has O(log b)

5) Bubble sort without loops

```
bool BSscan(vector<int>&myVec, int ind)
{ if(ind+1==myvec.size()) return false;
  bool ChangHap=false;
  if(myVec[ind]>myVec[ind+1])
    { swap(myvec[ind],myVec[ind+1]);
      ChangeHap=true;
    }
  return BSscan(myVec, ind+1)||ChangHap
}
void BSwOLoops(vector<int>& myvec)
{ bool ChangHap= BSscan(myvec,0);
  if (ChangHap)
     BSwOLoops(myvec);
  else
     return;
}
int main()
{ vector<int> myvec{4,2,1,9};
  BSwOLoops(myvec);
  print(myvec);
  return 0;
}
```

Dry Run:

e.g myVec = {4, 2, 1, 9}

∴ Call to BSWOLoops(myVec)

- Call to BSScan(myVec, 0)
  - 4 > 2 = True. Swap. ChaHap = T. {2, 4, 1, 9}
  - Call to BSScan(myVec, 1)
  - 4 > 1 = T. Swap. ChaHap = T. {2, 1, 4, 9}
- Call to BSScan(myVec, 2)
  - 4 > 9 = F. No swap. ChanHap = F
- ∴ Call to BSScan(myVec, 3)
  - Base case so return F. Backtrack
- Return T bec swap occurred
- return T " " "
  " T " " "

∴ Since overall, ChaHap = T, again call to BSWOLoops(myVec)

- Call to BSScan(myVec, 0):
  - 2 > 1 = T. Swap. ChaHap = T. {1, 2, 4, 9}
- Call to BSScan(myVec, 1):
  - 2 > 4 = F. No swap. ChaHap = F
- Call to BSScan(myVec, 2)
  - 4 > 9 = F. No swap
- Call to BSScan(myVec, 3)
  - base case → return false. Back track

- return false
- return T because swap occured
- " " " " "
- Because ChaHap=T, call to BSWO loops (myVec) again.
  - Call to BSScan(myVec,0)
    - 1>2=F No swap
  - ~~2>4~~ Call to BSScan(myVec,1)
    - 2>4=F. No swap
- Call (myVec,2)
  - 4>9=F. NO swap
- Call (myVec,3)
  - base case -> return false. Backtrack
- return F
- return F
- ChanHap = False so BSWO(myVec) not called again.

Callstack:

Tree

BSwOLoops(myVec)

BSscan(myVec, 0)

BSscan(myVec, 1)

BSscan(myVec, 2)

BSscan(myVec, 3)

BS(myVec)	{1, 2, 4, 9}
BS(myVec)	{2, 1, 4, 9}
BS(myVec)	{2 4 1 9}
BS(myVec)	{4, 2, 1, 9}

Time Complexity:

Time Complexity of BSscan is O(n) because it calls itself till it gets to vectors end. n is size of vector. BSwOlooops is $O(n^2)$ because it is called until swap keep happening. and for each swap, BSscan is also called so $O(n) * O(n) = O(n^2)$. So overall $O(n^2)$

3,4) Minor and Determinent

```
void Min(vector<vector<int>>& Mat,
    vector<vector<int>>& TMat, int p, int q,
    int size)
{ int i=0, j=0;
  for(int r=0; r<size; r++){
    for(int c=0; c<size; c++){
      if(r!=p && c!=q){
        TMat[i][j] = Mat[r][c];
        j++;
        if(j==size-1){
          i++;
          j=0;
        }
      }
    }
  }
}
```

```
int Det(vector<vector<int>> Mat, int size)
{ if(size==1) return Mat[0][0];
   int s=1;
   int d=0;
   vector<vector<int>> T(size-1, vector<int>(size-1));
   for(int x=0; x<size; x++){
      Min(Mat, T, 0, x, size);
      d = d + s * Mat[0][x] * Det(T, size-1);
      s = s * -1
   }
   return d;
}
int main()
{ vector<vector<int>> Mat { {1,-2,3},{2,0,3},
                            {1,5,4}};
   cout<< Det(Mat, 3);
   return 0;
}
```

Dry Run

e.g $Mat = \begin{bmatrix} 1 & -2 & 3 \\ 2 & 0 & 3 \\ 1 & 5 & 4 \end{bmatrix}$, size = 3

∴ Call to Det(Mat, 3)

• d = 0, s = +1, loop x = 0 to 2.

•• x = 0 :-

• Call to Min(Mat, T, 0, 0, 3). Fn turns T to $\begin{bmatrix} 0 & 3 \\ 5 & 4 \end{bmatrix}$

• Call to Det(T, 3-1=2)

• d = 0, s = 1, loop x = 0 to 1

•• x = 0 :-

Call to Min(T, T, 0, 0, 2), Gives [4]

Return Det(T, 2) :-

• d = d + s * Mat[0][0] * Det(T, 1)

= 0 + 1 * 0 * 4 = 0

s = -1;

•• x = 1

Call to Min(T, T, 0, 1, 2). Give [5]

Det(T, 1) gives 5

Return Det(T, 2)

•• d = 0 - 1 * 3 * 5 = -15

Return Det(Mat, 3)

d = 0 + 1 * 1 * (-15) = -15

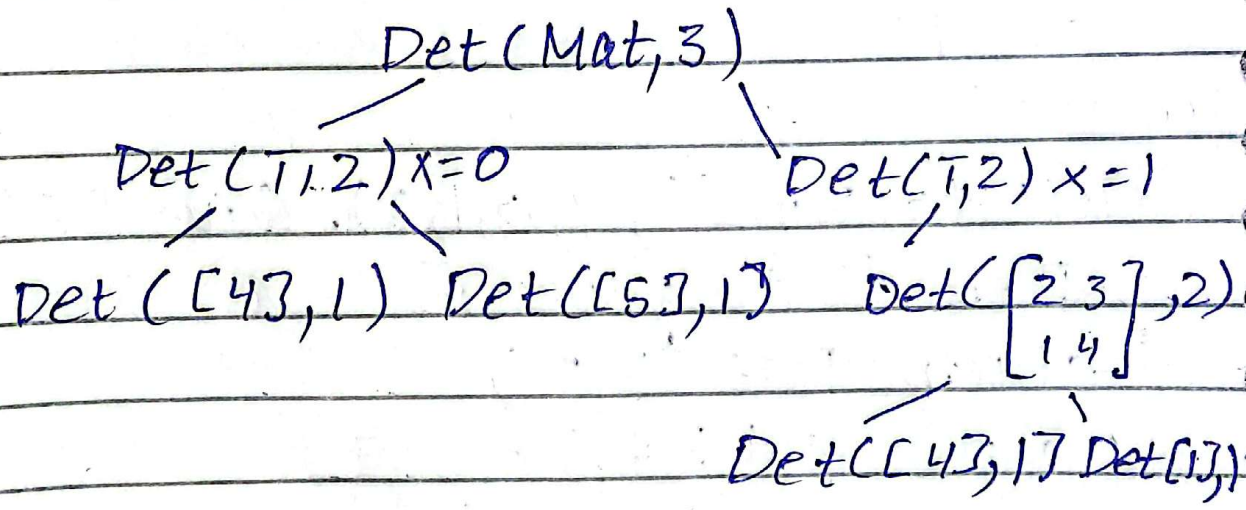
s = -1;

" X = 1 :-

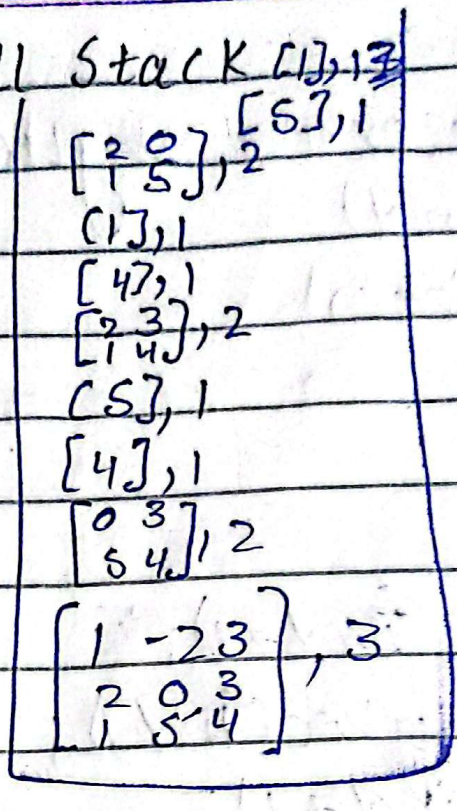
- Call to Min(Mat, T, 0, 1, 3).
  T is now $\begin{bmatrix} 2 & 3 \\ 1 & 4 \end{bmatrix}$.
- Call to Det (T, 2)

Same process is repeated as before. we find 3 Minors $\begin{bmatrix} 0 & 3 \\ 5 & 4 \end{bmatrix}$ $\begin{bmatrix} 2 & 3 \\ 1 & 4 \end{bmatrix}$ $\begin{bmatrix} 2 & 0 \\ 1 & 5 \end{bmatrix}$ and their determinents and then add them all.

Recursive Tree :-



Call Stack



Time Complexity :-

Time complexity of Min is $O(n^2)$ because it has nested loops. n is size of Mat.

Time complexity of Det is $O(n!)$ because in each recursive call, T[n-1][n-1] is created and Min fn is called.

6) Power Set

```
void Powset(vector<int>myvec,
    string S, int ind)
{ if(ind==myvec.size())
    {cout<<S;
     return;
    }
  Powset(myvec, S, ind+1);
  S=S+tostring(myvec[ind]);
  Powset(myvec, S, ind+1);
}
```

Dry run:-

e.g myvec = {1, 2}

∴ Call to Powset(myvec, "", 0)

- ind=0, S=""
- 0 != myvec.size

∴ Call to Powset(myvec, "", 1)

- ind=1, S="". 1 != myvec.size

∴ Call to Powset(myvec, "", 2)

ind=2, S="". 2 == 2 so print S.

we get {""} i.e empty set

Backtrack

- Powset(myvec, "", 1) : Call to Powset (myvec, "", 2)

∴ Powset(myVec, "2", 2)

• ind = 2, S = "2". 2 == myVec.size

so output S i.e {2} backtrack

∴ Powset(myVec, "", 0)

∴ Call to Powset(myVec, "1", 1)

• ind = 1, S = "1". 1 ≠ myVec.size

∴ Call to Powset(myVec, "1", 2)

ind = 2, S = "1". 2 == 2 so cout

S i.e {1}, backtrack

• Powset(myVec, "1", 1)

∴ Call Powset(myVec, "1,2", 2)

ind = 2, S = "1,2". 2 == 2 so

cout S i.e {1,2}. Return.

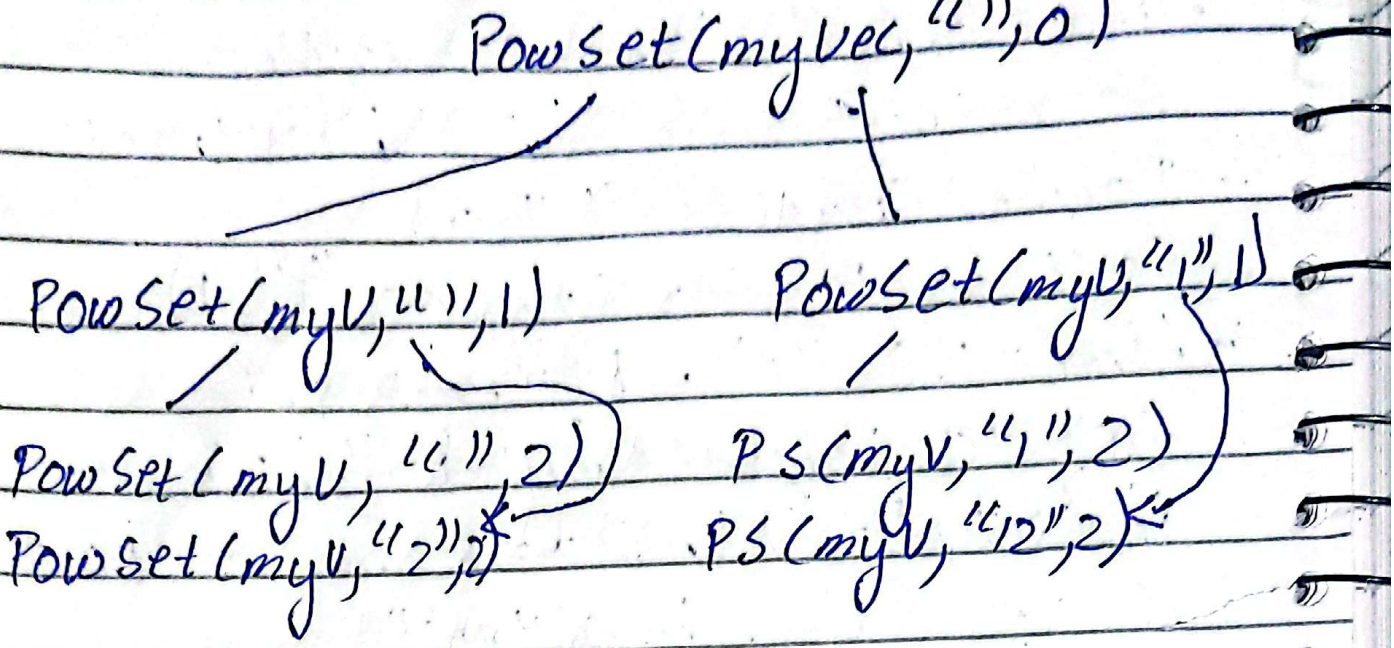
Call stack

Powset ({1,2}, "2", 2)

Powset ({1,2}, "", 1)

Powset ({1,2}, "", 0)

Call Stack & Tree



Time complexity :-

Since no. of Fn calls are 2^n because at each recursive call, 2 more fns are called. So overall is $O(2^n)$

8) Sum of first n terms of Arithmetic series

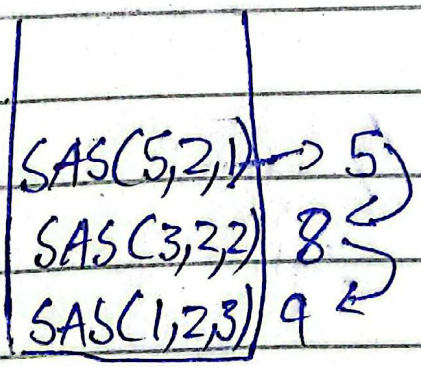
```
int SumAriSer(int al, int d, int N)
{ if (N==1) return al;
  else
      return al + SumAriser(al+d, d, N-1)
}
```

Dry run:-

e.g. SumAriSer(1, 2, 3) (al, d, N)

- N!=1 so return 1 + SAriSer(1+2, 2, 3-1) (3, 2)
- N!=1 so return 3 + SAriSer(3+2, 2, 2-1) (5, 1)
- 1==1 base case so return 5. Backtrack
- SAriSer(3, 2, 2) = 3+5 = 8
- SAriSer(1, 2, 3) = 1+8 = 9

Call Stack:-



Tree :-

SAS(1,2,3) 9
↘ 1+SAS(3,2,2) 8
↘ 3+SAS(5,2,1) 5
↘ return 5;

Time complexity :-

· For each call to SAS, a new call is made till base case i.e n==1 is reached. So overall time complexity is O(n).

7) 0-n bit Strings :-

```
void Zer to N bit S (queue<string> &Q,
    int N)
{ if(Q.size() ==0) return;
    string S = Q.front();
    Q.pop;
    cout<<S;
    if (S.size() <N)
{ Q.push (St "0");
  Q.push (St "1")
}
> Zer to N bit S( Q, N);
```

```
void ZerotoNbitsWrap(int N)
{ queue<string> Q;
  Q.push("0");
  Q.push("1");
  ZerotoNbits(Q,N);
}
```

Dry run:

e.g. Q = {"0", "1"}

"0" is popped, printed. then "00" "01" are added to Q.

Then "1" is popped & printed. "10", "11" will be pushed in Q. currently {00, 01, 10, 11}

These steps will repeat.

No further higher bit strings are added once the popped item has size equal to N.

Stack:-

// {} base case

keep poping front

//

// {001, 010, 011, 100, 101, 110, 111}

// {000, 001, 010, 011, 100, 101, 110, 111}

// {10, 11, 000, 011, 100, 101}

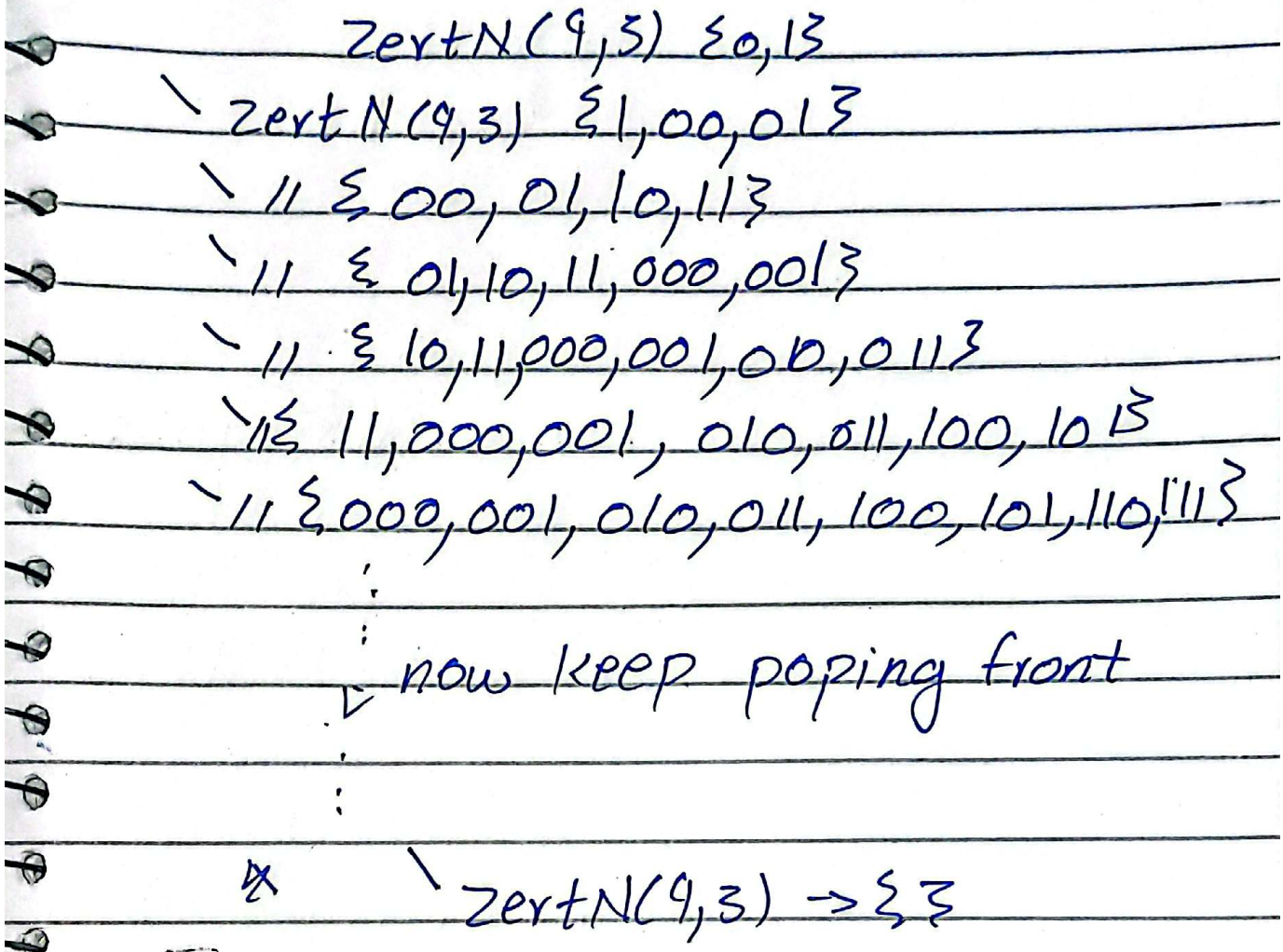
// {01, 10, 11, 000, 011}

// {00, 01, 10, 11}

// -> {1, 00, 01}

zero to N (2^3) -> {0, 1}

Tree:



Time complexity:-

The Fn calls itself recursively till queue is empty. At each call, two new string added to queue. So Overall we get $O(2^n)$